QUY HOẠCH ĐỘNG

qua 36 bài Timus

Lê Thành Nam

LÊ THÀNH NAM

MỤC LỤC

Trước khi bắt đầu **2**

Chương 1. Đếm trên dãy: trạng thái theo phần tử cuối **5**
Bài 1. Lá cờ (Timus 1225) 6
Bài 2. Số trong hệ cơ số K (Timus 1009) 12
Bài 3. Album đầu tay (Timus 2018) 17
Bài 4. Nhiếp ảnh gia khó tính (Timus 1260) 23
Bài 5. Số "ba nguyên tố" (Timus 1586) 28
Bài 6. Số trong hệ cơ số K - phiên bản 2 (Timus 1012) 33
Bài 7. Chuyện quả chanh (Timus 1513) 37

Chương 2. Đường đi trên lưới và trên đường thẳng **44**
Bài 8. Tàu điện ngầm (Timus 1119) 45
Bài 9. Tổng lớn nhất (Timus 1146) 51
Bài 10. Người làm vườn Kirill 3 (Timus 2072) 56
Bài 11. Bộ (Timus 1029) 62

Chương 3. Cái túi và quy hoạch động theo chữ số **69**
Bài 12. Hàm của Vasya với N một tỉ (Timus 1353) 70
Bài 13. Đất nước Hình Vuông (Timus 1073) 74
Bài 14. Cầu thang (Timus 1017) 79
Bài 15. Tổng chữ số (Timus 1658) 84
Bài 16. Vé may mắn (Timus 1036) 89
Bài 17. Một-hai, một-hai 2 (Timus 1495) 94

Chương 4. Chia dãy thành đoạn, dãy con và truy vết **99**
Bài 18. Ngựa hai màu (Timus 1167) 100
Bài 19. Cảm nhận cái đẹp (Timus 1501) 105
Bài 20. Dãy nhị phân theo thứ tự từ điển (Timus 1081) 111
Bài 21. Các đoạn thẳng (Timus 1078) 116
Bài 22. Tin nhắn SMS (Timus 1427) 120
Bài 23. Vé tàu hoả (Timus 1031) 124
Bài 24. Máy tẩy não (Timus 1900) 129

Chương 5. Quy hoạch động trên đoạn **134**
Bài 25. Ghi nhớ và xâu đối xứng (Timus 1635) 135
Bài 26. Dãy ngoặc (Timus 1183) 140
Bài 27. Gấp xâu (Timus 1238) 145

Chương 6. Quy hoạch động trên đồ thị có hướng không chu trình và trên cây **150**
Bài 28. Kẻ cuồng nhắn tin (Timus 1741) 151
Bài 29. Tiệc kỉ niệm (Timus 1039) 156
Bài 30. Cây táo nhị phân (Timus 1018) 161

Chương 7. Kĩ thuật nâng cao: ma trận, bitmask, xác suất, đếm tổ hợp **166**
Bài 31. Số trong hệ cơ số K - phiên bản 3 (Timus 1013) 167
Bài 32. Đại bàng trên mái nhà (Timus 1223) 171
Bài 33. Nắp chai (Timus 1326) 175
Bài 34. Pháo hoa kỉ niệm (Timus 1776) 180
Bài 35. Tuyến tàu biển (Timus 1172) 185
Bài 36. Lương cho người máy (Timus 1696) 190

Lê Thanh Nam

GIỚI THIỆU

Cuốn sách này dành cho ai

Cuốn sách gồm 36 bài quy hoạch động chọn từ kho đề Timus Online Judge (acm.timus.ru) của Đại học Liên bang Ural (Nga), xếp từ dễ đến khó. Bài dễ nhất dành cho học sinh mới làm quen với quy hoạch động. Bài khó nhất ngang tầm đề chọn đội tuyển quốc gia.

Mỗi bài gồm hai phần. **Đề bài** được viết lại bằng tiếng Việt, giữ đúng yêu cầu, ràng buộc, định dạng vào/ra và ví dụ của đề gốc. **Hướng dẫn giải** đi theo mạch quen thuộc: Ngây thơ → bế tắc → lóe sáng → tối ưu, mỗi bước đều có hình minh hoạ.

> **Về cách viết đề**
> *Đề trong sách không phải bản dịch từng câu. Mỗi đề được viết lại theo cách hiểu tự nhiên của người Việt, bỏ những chi tiết kể chuyện không ảnh hưởng đến lời giải. Mọi điều kiện, giới hạn và định dạng vào/ra được giữ nguyên. Ở những chỗ đề gốc nói chưa rõ, sách ghi chú cách hiểu đã được kiểm chứng qua ví dụ. Mỗi bài có số hiệu Timus để học sinh mở đề gốc và nộp bài.*

CÁCH GIẢI

Hướng dẫn giải của mỗi bài chia thành sáu phần, mỗi phần có một dải màu riêng để dễ nhận ra. Trong mỗi phần, lời giải đi từng bước nhỏ (Bước 1, Bước 2, …), và hầu hết các bước đều có hình minh hoạ.

Phần	Nội dung
HIỂU ĐỀ	Các trường hợp đúng, sai được vẽ ra để hiểu chính xác yêu cầu của đề
NGÂY THƠ	Ý tưởng tự nhiên nhất, sơ đồ khối, hình chạy tay và code vét cạn
BẾ TẮC	Thời gian chạy đo thật của code vét cạn, chỉ ra nguyên nhân chậm bằng hình, kết thúc bằng một câu hỏi then chốt
LÓE SÁNG	Đặt bài toán con, chia trường hợp có hình, rút ra công thức truy hồi, cơ sở và thứ tự tính
TỐI ƯU	Hình điền bảng từng ô, code tối ưu và bảng so sánh với cách vét cạn
LƯU Ý	Những chỗ dễ sai và phần mở rộng

Các thời gian chạy trong phần Bế tắc được đo thật trên máy tính thông thường; trên máy chấm Timus con số có thể khác đôi chút, nhưng xu hướng tăng là như nhau.

SỬ DỤNG SÁCH

Với mỗi bài, học sinh nên đọc đề, tự nghĩ ít nhất 20-30 phút rồi mới đọc hướng dẫn. Khi đọc hướng dẫn, nên dừng sau phần "Bế tắc" và thử tự tìm hướng tối ưu trước khi đọc tiếp.

Mọi code trong sách đã được chạy với ví dụ của đề và đối chiếu với lời giải vét cạn trên hàng nghìn test ngẫu nhiên. Code tối ưu cũng đã được đo thời gian trên test lớn nhất theo ràng buộc.

Lê Thành Nam

Cuối mỗi đề có ghi **độ khó Timus**. Đây là thang điểm của Timus, tính theo số người giải được: số càng nhỏ thì bài càng dễ. Trong sách, độ khó trải từ 35 đến 860.

Nộp bài trên Timus

• Đăng kí tài khoản ở mục **Register** trên trang chủ Timus. Timus cấp cho mỗi người một mã định danh (Judge ID) dùng khi nộp bài.

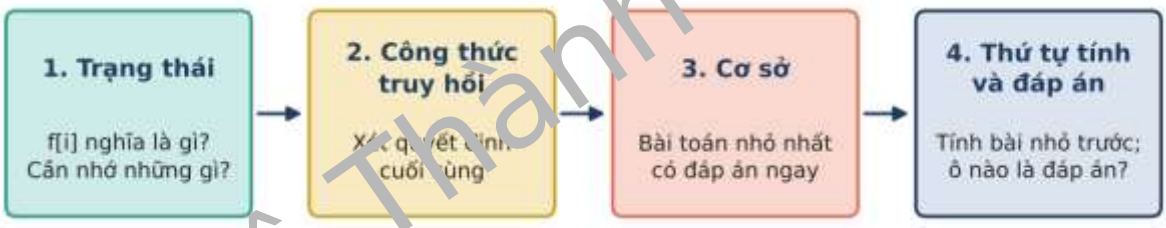
• Mở đề theo số hiệu, chọn **Submit solution**, chọn trình biên dịch C++ (ví dụ G++ 13 hoặc Visual C++), dán code và nộp.

• Chương trình đọc dữ liệu từ bàn phím (stdin) và ghi ra màn hình (stdout). Không dùng `freopen` để mở tệp.

• Kết quả **Accepted** nghĩa là bài đúng. Các kết quả thường gặp khác gồm Wrong answer (sai đáp án), Time limit exceeded (quá thời gian) và Memory limit exceeded (quá bộ nhớ). Timus cho biết bài sai ở test thứ mấy.

04 bước giải của Quy hoạch động

Mọi bài trong sách đều được giải theo cùng bốn bước. Học thuộc lòng bốn bước này quan trọng hơn học thuộc từng bài.



Hình 0.1. Bốn bước xây dựng lời giải quy hoạch động

• **Bước 1. Trạng thái.** Chọn một bộ thông tin nhỏ gọn mô tả "bài toán con". Ví dụ: f[i] là số cách xếp i sọc cờ. Câu hỏi then chốt: cần nhớ những gì về phần đã làm để quyết định tiếp phần còn lại?

• **Bước 2. Công thức truy hồi.** Viết f của bài toán lớn theo f của các bài toán nhỏ hơn. Cách nghĩ hiệu quả nhất là xét **quyết định cuối cùng**: phần tử cuối là gì, đoạn cuối dài bao nhiêu, lần cuối chọn món nào.

• **Bước 3. Cơ sở.** Xác định các bài toán nhỏ nhất có đáp án ngay, không cần truy hồi.

• **Bước 4. Thứ tự tính và đáp án.** Tính sao cho khi cần f của bài toán nhỏ thì nó đã có sẵn. Cuối cùng chỉ ra ô nào của bảng là đáp án.

Nhận biết một bài có thể dùng quy hoạch động qua hai dấu hiệu. Thứ nhất, lời giải ngây thơ (vét cạn) tính đi tính lại cùng một bài toán con. Thứ hai, đáp án tối ưu của bài toán lớn được ghép từ đáp án tối ưu của các bài toán con.

Lê Thanh Nam

Bố cục sách

Chương	Chủ đề	Bài (số hiệu Timus)
1	Đếm trên dãy: trạng thái theo phần tử cuối	1225, 1009, 2018, 1260, 1586, 1012, 1513
2	Đường đi trên lưới và trên đường thẳng	1119, 1146, 2072, 1029
3	Cái túi và quy hoạch động theo chữ số	1353, 1073, 1017, 1658, 1036, 1495
4	Chia dãy thành đoạn, dãy con, truy vết	1167, 1501, 1081, 1078, 1427, 1031, 1900
5	Quy hoạch động trên đoạn	1635, 1183, 1238
6	Quy hoạch động trên đồ thị có hướng không chu trình và trên cây	1741, 1039, 1018
7	Kĩ thuật nâng cao: ma trận, bitmask, xác suất, đếm tổ hợp	1013, 1223, 1326, 1776, 1172, 1696

Lê Thành Nam

Chương 1. Đếm trên dãy: Trạng thái theo phần tử cuối

Trọng tâm chương

Các bài trong chương này đều yêu cầu đếm số dãy thoả một điều kiện "cục bộ": điều kiện chỉ liên quan đến vài phần tử đứng cạnh nhau. Kĩ thuật chung là **đặt trạng thái theo phần đuôi của dãy**: chỉ cần nhớ vài phần tử cuối (hoặc độ dài đoạn lặp cuối) là đủ biết phần tử tiếp theo được phép là gì. Đáp án thường lớn rất nhanh, nên chương này cũng luyện cách xử lí số lớn và phép lấy dư.

Các bài trong chương

Bài	Tên bài	Số hiệu Timus	Độ khó
1	Lá cờ (Flags)	1225	35
2	Số trong hệ cơ số K (K-based Numbers)	1009	95
3	Album đầu tay (The Debut Album)	2018	102
4	Nhiếp ảnh gia khó tính (Nudnik Photographer)	1260	112
5	Số "ba nguyên tố" (Threeprime Numbers)	1586	144
6	Số trong hệ cơ số K - phiên bản 2 (K-based Numbers. Version 2)	1012	181
7	Chuyện quả chanh (Lemon Tale)	1513	381

Lê Thành Nam

Bài 1. Lá cờ

Timus 1225 - Flags Thời gian: 1,0 giây · Bộ nhớ: 64 MB · Độ khó Timus: 35
Đề gốc: https://acm.timus.ru/problem.aspx?space=1&num=1225

Đề bài

Nhân ngày Quốc kì Nga, chủ một cửa hàng muốn trang trí tủ kính bằng N sọc vải xếp thành một dải. Mỗi sọc có một trong ba màu: trắng, xanh hoặc đỏ. Ông đặt ra hai yêu cầu:

- Hai sọc đứng cạnh nhau không được cùng màu.
- Mỗi sọc xanh phải nằm giữa hai sọc khác màu nhau, tức một bên là sọc trắng và bên kia là sọc đỏ.

Hãy đếm số cách xếp thoả mãn cả hai yêu cầu.

Dữ liệu vào

Một số nguyên N - Số sọc vải.

Kết quả

Một số nguyên - Số cách xếp.

Ràng buộc

- $1 \le N \le 45$.

Ví dụ

Dữ liệu vào	Kết quả
1	2
2	2
3	4

Giải thích ví dụ

Kí hiệu T, X, Đ lần lượt là trắng, xanh, đỏ. Với N = 3 có bốn cách: TĐT, ĐTĐ, TXĐ, ĐXT. Với N = 1 hoặc N = 2 không thể có sọc xanh, vì sọc xanh cần hai hàng xóm.

Lê Thành Nam

Hướng dẫn giải

HIỂU ĐỀ Hiểu đúng đề bài

Kí hiệu T là sọc trắng, X là sọc xanh, Đ là sọc đỏ. Hình dưới đây cho vài dải 3 sọc và giải thích vì sao dải đó hợp lệ hay không.



Hình 1.1. *Một số dải 3 sọc hợp lệ (✓) và không hợp lệ (✗)*

Từ yêu cầu thứ hai, rút ra hai nhận xét sẽ dùng nhiều lần:

- Sọc xanh **không bao giờ đứng ở hai đầu dải**, vì ở đầu dải nó chỉ có một hàng xóm.
- Vì vậy **sọc cuối cùng luôn là trắng hoặc đỏ**.

Với N = 1 hoặc N = 2 không thể có sọc xanh, nên chỉ có 2 cách: T, Đ (khi N = 1) hoặc TĐ, ĐT (khi N = 2).

NGÂY THƠ Ý tưởng ngây thơ: thử hết mọi cách tô màu

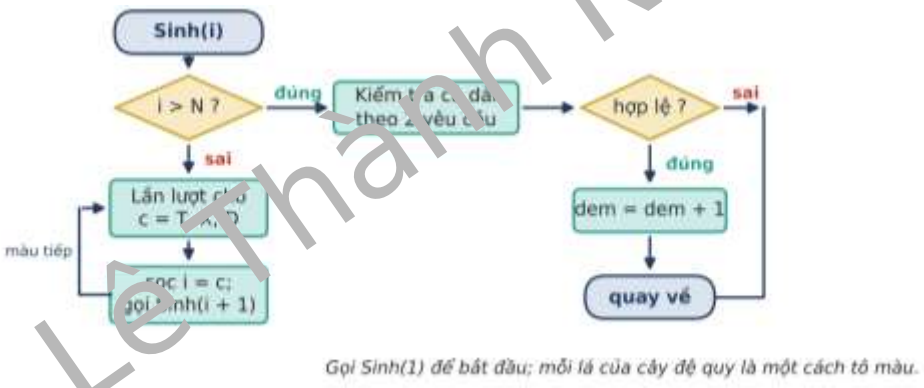
Cách nghĩ đầu tiên của hầu hết học sinh: **tô thử tất cả các cách, cách nào đúng thì đếm**. Mỗi sọc có 3 màu, nên N sọc có $3 \times 3 \times \ldots \times 3 = 3^N$ cách tô.

Bước 1. Sinh mọi cách tô màu bằng đệ quy

Viết hàm Sinh(i) tô sọc thứ i. Lần lượt thử ba màu T, X, Đ cho sọc i; với mỗi màu lại gọi Sinh(i + 1) để tô sọc tiếp theo.

Bước 2. Khi tô đủ N sọc thì kiểm tra

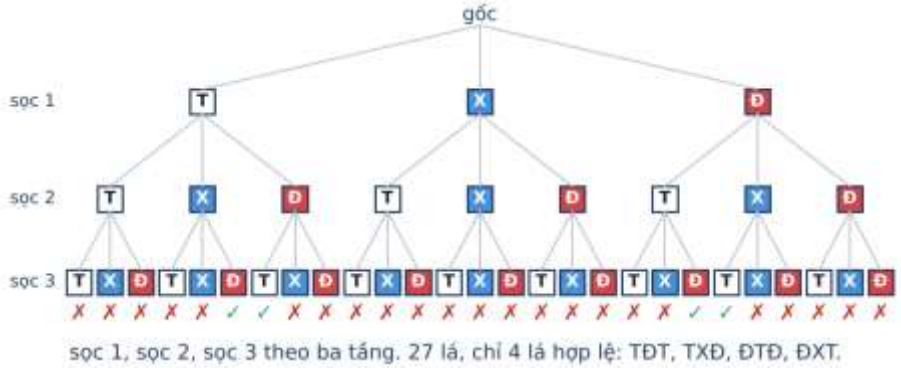
Khi i > N, cả dải đã được tô. Kiểm tra hai yêu cầu: không có hai sọc kề nhau cùng màu, và mỗi sọc xanh có hai hàng xóm khác màu nhau. Nếu đúng thì tăng biến đếm.



Hình 1.2. Sơ đồ khối của thuật toán vét cạn

Bước 3. Chạy tay với N = 3

Cây đệ quy có 3 tầng ứng với 3 sọc. Mỗi nút có 3 nhánh ứng với 3 màu, nên tầng cuối có $3^3 = 27$ lá. Kiểm tra từng lá, chỉ có 4 lá hợp lệ, đúng bằng đáp án của ví dụ.



Hình 1.3. Cây đệ quy khi N = 3: 27 cách tô, 4 cách hợp lệ

Lê Thanh Nam

Bước 4. Cài đặt

Code. Vét cạn 3^N cách tô màu

```
#include <bits/stdc++.h>
using namespace std;
int n;
long long cnt;
string s;
bool ok(){
    for (int i = 0; i + 1 < n; i++) if (s[i] == s[i + 1]) return false;
    for (int i = 0; i < n; i++)
        if (s[i] == 'X') {
            if (i == 0 || i == n - 1) return false;
            if (s[i - 1] == s[i + 1]) return false;
        }
    return true;
}
void gen(int i){
    if (i == n) { if (ok()) cnt++; return; }
    for (char c : string("TXD")) { s[i] = c; gen(i + 1); }
}
int main(){
    cin >> n;
    s.assign(n, ' ');
    gen(0);
    cout << cnt << "\n";
}
```

Hàm ok() kiểm tra một dải đã tô xong, mất N bước. Tổng thời gian là $3^N \cdot N$.

BẾ TẮC Bế tắc: vì sao vét cạn không dùng được

Bước 1. Đo thời gian thật

Chạy chương trình vét cạn với các giá trị N khác nhau và đo thời gian:



Hình 1.4. Thời gian chạy của code vét cạn tăng gấp 3 lần mỗi khi N tăng thêm 1

Chỉ với N = 19, chương trình đã chạy 6,5 giây, vượt xa giới hạn 1 giây. Theo đà tăng gấp 3 lần, với N = 45 cần khoảng 520 nghìn năm.

Bước 2. Tìm nguyên nhân: tính đi tính lại những việc giống nhau

Hãy nhìn hai cách tô 3 sọc đầu: TĐT và ĐXT. Chúng khác nhau, nhưng cùng kết thúc bằng sọc trắng. Sọc thứ 4 trở đi chỉ phụ thuộc vào sọc cuối cùng, nên mọi cách tô tiếp theo của hai dải này **giống hệt nhau**. Thế nhưng vét cạn vẫn duyệt lại phần đuôi cho từng dải một.



Hình 1.5. Hai dải khác nhau nhưng có cùng "tương lai"

Lê Thành Nam

Câu hỏi then chốt

Để biết sọc tiếp theo được tô màu gì, ta có cần nhớ **toàn bộ** dải đã tô không? Hay chỉ cần nhớ một vài sọc cuối?

LÓE SÁNG Lóe sáng: chỉ cần nhìn vào sọc gần cuối

Bước 1. Đặt tên cho bài toán con

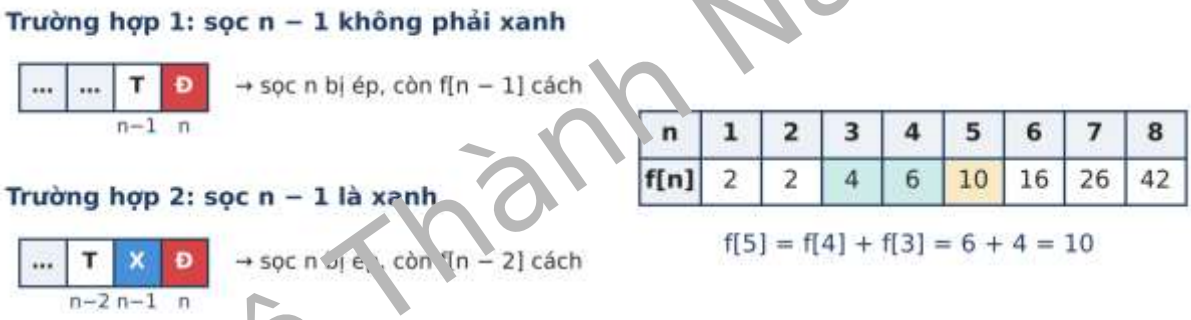
Gọi **f[n]** là số cách xếp hợp lệ n sọc. Đề bài hỏi f[N]. Ta tìm cách tính f[n] từ các giá trị f nhỏ hơn.

Bước 2. Sọc cuối là trắng hoặc đỏ - vậy nhìn sọc ngay trước nó

Theo phần "Hiểu đúng đề bài", sọc thứ n luôn là T hoặc Đ. Có hai trường hợp cho sọc thứ n - 1:

• **Trường hợp 1: sọc n - 1 không phải xanh** (là T hoặc Đ). Vì hai sọc kề nhau phải khác màu, sọc n bắt buộc là màu còn lại trong T, Đ. Như vậy sọc n bị "ép" hoàn toàn: mỗi cách xếp hợp lệ n - 1 sọc cho đúng một cách xếp n sọc. Có **f[n - 1]** cách.

• **Trường hợp 2: sọc n - 1 là xanh.** Khi đó sọc n - 2 là T hoặc Đ, và sọc n phải khác màu sọc n - 2 (để sọc xanh nằm giữa trắng và đỏ). Sọc n - 1 và sọc n đều bị ép: mỗi cách xếp hợp lệ n - 2 sọc cho đúng một cách. Có **f[n - 2]** cách.



Hình 1.6. Hai trường hợp của sọc n - 1

Lê Thành Nam

Bước 3. Viết công thức truy hồi

Cộng hai trường hợp:

$$f[n] = f[n - 1] + f[n - 2]$$

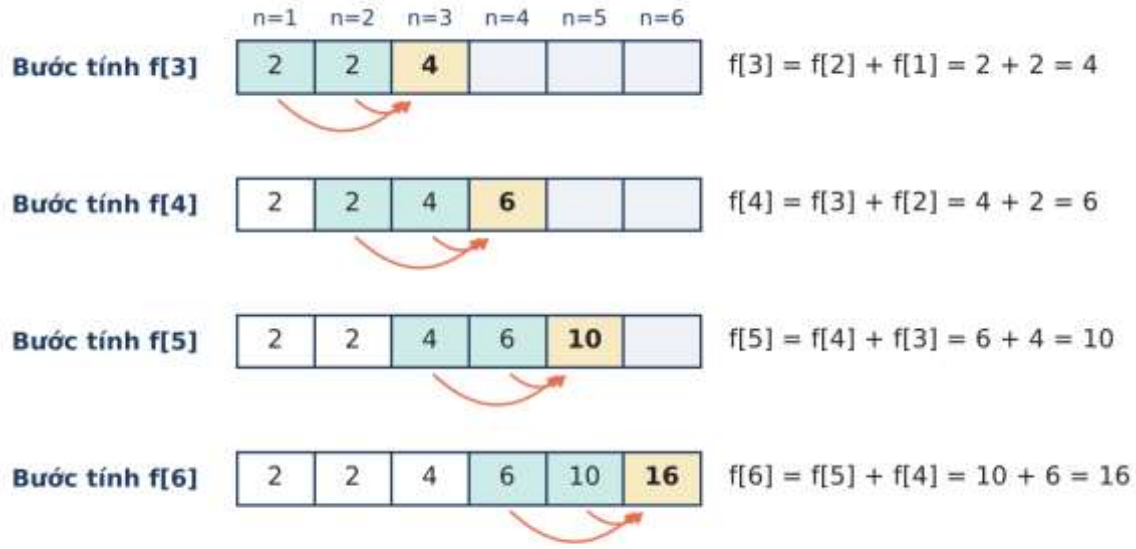
Bước 4. Cơ sở và thứ tự tính

Hai giá trị nhỏ nhất tính trực tiếp: f[1] = 2 và f[2] = 2. Từ đó tính lần lượt f[3], f[4], …, f[N] theo thứ tự tăng dần. Khi tính f[n], hai giá trị f[n - 1] và f[n - 2] đã có sẵn.

TỐI ƯU Lời giải tối ưu

Bước 1. Chạy tay điền bảng

Mỗi ô mới bằng tổng hai ô liền trước (tô màu xanh trong hình).



Hình 1.7. Điền bảng f từ trái sang phải

Kiểm tra: f[3] = 4 đúng bằng ví dụ của đề. Đây chính là dãy Fibonacci nhảy đôi.

Bước 2. Cài đặt

Code. Lời giải O(N)

```
#include <bits/stdc++.h>
using namespace std;
int main(){
    ios_base::sync_with_stdio(false); cin.tie(nullptr);
    int n; cin >> n;
    vector<long long> f(max(n + 1, 3));
    f[1] = 2; f[2] = 2;
    for (int i = 3; i <= n; i++) f[i] = f[i - 1] + f[i - 2];
    cout << f[n] << "\n";
}
```

Bước 3. So sánh hai cách giải

Cách giải	Số phép tính	N = 45	Kết quả
Vét cạn	$3^N \cdot N$	$\approx 10^{23}$	quá thời gian
Quy hoạch động	N	45	chạy tức thì

Hình 1.8. Vét cạn và quy hoạch động

LƯU Ý Những chỗ dễ sai

- f[45] = 2 269 806 340 vượt giới hạn kiểu int (khoảng $2{,}1 \cdot 10^9$), nên phải dùng long long.
- Với N = 1, mảng phải có ít nhất 3 phần tử để gán f[2] không bị tràn chỉ số. Code dùng `max(n + 1, 3)` để tránh lỗi này.
- Đáp án với N = 1 và N = 2 đều là 2. Nhiều bạn trả lời nhầm là 3 và 9 vì quên điều kiện.

Lê Thành Nam

Bài 2. Số trong hệ cơ số K

Timus 1009 - K-based Numbers Thời gian: 0,5 giây · Bộ nhớ: 64 MB · Độ khó Timus: 95
Đề gốc: https://acm.timus.ru/problem.aspx?space=1&num=1009

Đề bài

Xét các số có đúng N chữ số khi viết trong hệ cơ số K. Chữ số đầu tiên phải khác 0, vì nếu không thì số đó có ít hơn N chữ số. Một số gọi là **hợp lệ** nếu khi viết trong hệ cơ số K, nó không chứa hai chữ số 0 đứng liền nhau.

Ví dụ trong hệ thập phân: 1010230 là số hợp lệ có 7 chữ số; 1000198 không hợp lệ vì chứa hai chữ số 0 liền nhau; còn 0001235 thực chất là số có 4 chữ số.

Cho N và K, hãy đếm số các số hợp lệ có N chữ số.

Dữ liệu vào

Hai dòng: dòng đầu là N, dòng thứ hai là K (viết trong hệ thập phân).

Kết quả

Số các số hợp lệ, viết trong hệ thập phân.

Ràng buộc

- $2 \le K \le 10;\ N \ge 2;\ N + K \le 18.$

Ví dụ

Dữ liệu vào	Kết quả
2 10	90

Giải thích ví dụ

Có 90 số có hai chữ số (từ 10 đến 99) và không số nào chứa "00", nên đáp án là 90.

Lê Thành Nam

Hướng dẫn giải

HIỂU ĐỀ Hiểu đúng đề bài

"Số có đúng N chữ số trong hệ cơ số K" là một dãy N chữ số, mỗi chữ số từ 0 đến K - 1, và **chữ số đầu khác 0**. Số hợp lệ khi trong dãy không có hai chữ số 0 đứng cạnh nhau.

Số	Hệ cơ số	Hợp lệ?	Lí do
1010230	10	✓	không có hai chữ số 0 liền nhau
1000198	10	✗	chứa "00"
0001235	10	✗	thực chất chỉ có 4 chữ số
102	3	✓	chữ số đầu khác 0, không có "00"
200	3	✗	chứa "00"

Hình 1.9. Vài số hợp lệ và không hợp lệ